

INFO145 - Diseño y Análisis de Algoritmos

8 - Entropía y Códigos de Huffman

Héctor Ferrada
académico

Instituto de Informática
Universidad Austral de Chile

Entropy

La Entropía es un concepto utilizado en Teoría de la Información:

- En Data Compression se usa la entropía para determinar la complejidad de comprimir un stream.
- **La Entropía** de un objeto χ es, en términos amplios, la **mínima cantidad de bits** requeridos para representar univocamente a χ desde un conjunto.
- Por tanto, la entropía es usada como una **cota inferior** al espacio usado para cualquier **representación comprimida** de un objeto.

Worst Case Entropy

Suponga que deseamos asignar un código a un elemento χ proveniente de un universo $\mathcal{U}$.

- La entropía nos dice cual es la **mínima cantidad de bits** para asignar un código para χ en $\mathcal{U}$, es decir su representación o identificador.

Worst Case Entropy

Suponga que deseamos asignar un código a un elemento χ proveniente de un universo $\mathcal{U}$.

- La entropía nos dice cual es la **mínima cantidad de bits** para asignar un código para χ en $\mathcal{U}$, es decir su representación o identificador.
- Si solo es posible asignar códigos del mismo largo a los elementos de $\mathcal{U}$, entonces corresponde **al peor caso posible** en términos de compresión.
- Un principio en compresión es: **asignar códigos cortos a elementos muy probables, dejando los códigos más largos a elementos menos probables**

Worst Case Entropy

Suponga que deseamos asignar un código a un elemento χ proveniente de un universo $\mathcal{U}$.

- La entropía nos dice cual es la **mínima cantidad de bits** para asignar un código para χ en $\mathcal{U}$, es decir su representación o identificador.
- Si solo es posible asignar códigos del mismo largo a los elementos de $\mathcal{U}$, entonces corresponde **al peor caso posible** en términos de compresión.
- Un principio en compresión es: **asignar códigos cortos a elementos muy probables, dejando los códigos más largos a elementos menos probables**
- Así, si todos los elementos son igualmente probables, todos tendrán códigos del mismo largo, requiriendo la misma cantidad de bits (ej. en secuencias aleatorias). Esta se llama **Entropía de Peor Caso**, y viene dada por:

$$\mathcal{H}_{wc}(\mathcal{U}) = \lg |\mathcal{U}| \text{ bits.}$$

Donde $\lg$ denota el logaritmo en base 2.

Worst Case Entropy

$$\mathcal{H}_{WC}(\mathcal{U}) = \lg |\mathcal{U}| \text{ bits.}$$

- Note que si usamos codigos de largo $\ell < \mathcal{H}_{WC}(\mathcal{U})$, tendríamos solo $2^\ell < 2^{\mathcal{H}_{WC}(\mathcal{U})} = 2^{\lg |\mathcal{U}|} = |\mathcal{U}|$ códigos $\Rightarrow$ no habrían códigos para todos los elementos de $\mathcal{U}$.
- Inclusive, si los códigos fuesen de largo variable, aún así el código de largo mayor debe tener almenos $\mathcal{H}_{WC}(\mathcal{U})$ bits.

Worst Case Entropy

$$\mathcal{H}_{WC}(\mathcal{U}) = \lg |\mathcal{U}| \text{ bits.}$$

- Note que si usamos codigos de largo $\ell < \mathcal{H}_{WC}(\mathcal{U})$, tendríamos solo $2^\ell < 2^{\mathcal{H}_{WC}(\mathcal{U})} = 2^{\lg |\mathcal{U}|} = |\mathcal{U}|$ códigos $\Rightarrow$ no habrían códigos para todos los elementos de $\mathcal{U}$.
- Inclusive, si los códigos fuesen de largo variable, aún así el código de largo mayor debe tener almenos $\mathcal{H}_{WC}(\mathcal{U})$ bits.

Ejemplos:

- La $\mathcal{H}_{WC}$ para **todas las secuencias binarias de largo n** es $\lg(2^n) = n$ bits.
- Ahora para **todos los strings de largo n** provenientes de un universo Σ de tamaño $|\Sigma| = \sigma$. $\mathcal{H}_{WC} = \lg(\sigma^n) = n \lg \sigma$.

Shannon Entropy

- En teoría de la información, se asume que existe un origen infinito que emite los elementos μ de $\mathcal{U}$ con probabilidades $Pr(\mu)$.
- Si fuese posible usar códigos de largo variable, asignando los más cortos a los más frecuentes, el largo promedio se reduce a:

$$\sum_{\mu \in \mathcal{U}} Pr(\mu) \cdot \ell(\mu), \text{ donde } \ell(u) \text{ es el largo del código para } \mu$$

Resultado fundamental de Shannon:

Si cada elemento $\mu \in \mathcal{U}$ tiene probabilidad $Pr(\mu)$, la **Entropía de Shannon de la Distribución de Probabilidad** $Pr : \mathcal{U} \rightarrow [0.0, 1.0]$, viene dada por:

$$\mathcal{H}(Pr) = \sum_{\mu \in \mathcal{U}} Pr(\mu) \cdot \lg \frac{1}{Pr(\mu)}$$

Es decir, Shannon sugiere que **el largo del código de μ sea** $\lg \frac{1}{Pr(\mu)}$.

Shannon Entropy

$$\mathcal{H}(Pr) = \sum_{\mu \in \mathcal{U}} Pr(\mu) \cdot \lg \frac{1}{Pr(\mu)}$$

- $\mathcal{H}(Pr)$ puede ser interpretado como la **cantidad de bits de información contenidos en los elementos de $\mathcal{U}$** .
- Así logramos que el largo del código de un elemento μ sea **indirectamente proporcional a su probabilidad de ocurrencia**.
- Note que si un elemento μ tiene $Pr(\mu)$ muy cercana a 1, entonces $\mathcal{H}(Pr)$ será muy cercana a 0.

Ej., sea una $Pr(\mathcal{U})$ para las 5 actividades que hacen los alumnos al interior de la UACH = {en clases(60%), estudiando(20%), en recreación(15%), en pruebas(2.5%), en tramites(2.5%)}. Calcule $\mathcal{H}(Pr)$

Shannon Entropy

$$\mathcal{H}(Pr) = \sum_{\mu \in \mathcal{U}} Pr(\mu) \cdot \lg \frac{1}{Pr(\mu)}$$

- $\mathcal{H}(Pr)$ puede ser interpretado como la **cantidad de bits de información contenidos en los elementos de $\mathcal{U}$** .
- Así logramos que el largo del código de un elemento μ sea **indirectamente proporcional a su probabilidad de ocurrencia**.
- Note que si un elemento μ tiene $Pr(\mu)$ muy cercana a 1, entonces $\mathcal{H}(Pr)$ será muy cercana a 0.

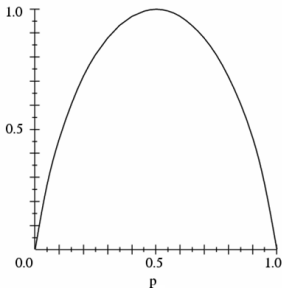
Ej., sea una $Pr(\mathcal{U})$ para las 5 actividades que hacen los alumnos al interior de la UACH = {en clases(60%), estudiando(20%), en recreación(15%), en pruebas(2.5%), en tramites(2.5%)}. Calcule $\mathcal{H}(Pr)$

$$\begin{aligned}\mathcal{H}(Pr) &= 0.6 \cdot \lg \frac{1}{0.6} + 0.2 \cdot \lg \frac{1}{0.2} + 0.15 \cdot \lg \frac{1}{0.15} + 2 \cdot 0.025 \cdot \lg \frac{1}{0.025} = \\ &= 0.6 \cdot 0.737 + 0.2 \cdot 2.322 + 0.15 \cdot 2.737 + 2 \cdot 0.025 \cdot 5.322 = 1.583\end{aligned}$$

Entropía Binaria

Suponga que trabajamos con bits y la fuente de origen emite 1's con probabilidad p y 0's con probabilidad $1 - p$. La entropía de Shannon, aquí llamada **Entropía Binaria**, viene dada por:

$$\mathcal{H}(p) = p \cdot \lg \frac{1}{p} + (1 - p) \cdot \lg \frac{1}{1-p}$$



En la imagen, la entropía es el eje Y ; donde, $\mathcal{H}(p) = 0$ cuando $p = 0$ o $p = 1$; y $\mathcal{H}(p) = 1$ cuando $p = 1 - p = 1/2$.

$\Rightarrow$ A mayor incertidumbre mayor es la entropía

Entropía para Secuencias de Símbolos

Dado un origen “infinito” que emite símbolos provenientes de un alfabeto $\Sigma = \{s_1, s_2, \dots, s_\sigma\}$, donde cada símbolo s tiene probabilidad P_s , entonces la **Entropía de Shannon para dicho origen** es:

$$0 \leq \mathcal{H}(P_1, P_2, \dots, P_\sigma) = \sum_{s_1 \leq s \leq s_\sigma} P_s \lg \frac{1}{P_s}$$

Así, para un secuencia finita $S[1..n] \in \Sigma^n$, donde n_i es la cantidad de veces que el i -ésimo símbolo aparece en S ; la **Entropía de Shannon para S** es:

$$\mathcal{H}(S) = \mathcal{H}\left(\frac{n_1}{n}, \frac{n_2}{n}, \dots, \frac{n_\sigma}{n}\right) = \sum_{1 \leq s \leq \sigma} \frac{n_s}{n} \lg \frac{n}{n_s}$$

Ejemplo. calcular $\mathcal{H}(S)$ para la secuencia $S = \text{carretera} \dots$

Entropía para Secuencias de Símbolos

Dado un origen “infinito” que emite símbolos provenientes de un alfabeto $\Sigma = \{s_1, s_2, \dots, s_\sigma\}$, donde cada símbolo s tiene probabilidad P_s , entonces la **Entropía de Shannon para dicho origen** es:

$$0 \leq \mathcal{H}(P_1, P_2, \dots, P_\sigma) = \sum_{s_1 \leq s \leq s_\sigma} P_s \lg \frac{1}{P_s}$$

Así, para un secuencia finita $S[1..n] \in \Sigma^n$, donde n_i es la cantidad de veces que el i -ésimo símbolo aparece en S ; la **Entropía de Shannon para S** es:

$$\mathcal{H}(S) = \mathcal{H}\left(\frac{n_1}{n}, \frac{n_2}{n}, \dots, \frac{n_\sigma}{n}\right) = \sum_{1 \leq s \leq \sigma} \frac{n_s}{n} \lg \frac{n}{n_s}$$

Ejemplo. calcular $\mathcal{H}(S)$ para la secuencia $S = \text{carretera} \dots$

$$\mathcal{H}(S) = 2 \frac{1}{9} \lg 9 + 2 \frac{2}{9} \lg \frac{9}{2} + \frac{1}{3} \lg 3 \simeq 2.197.$$

Por tanto podríamos comprimir S , usando un compresor que trate a cada símbolo de forma independiente, hasta $n\mathcal{H}(S) = 19.773$ bits.

Coding

Del concepto de entropía, tenemos que:

- La entendemos como la mínima cantidad de bits necesarios para representar unívocamente un objeto desde su conjunto $\mathcal{U}$.
- Por tanto, nos da el largo promedio mínimo al cuál podemos optar asignando códigos fijos a los elementos del alfabeto de origen Σ .
- Inclusive, para cada símbolo s , nos sugiere usar como largo óptimo $\lg \frac{1}{Pr(s)}$ bits.

Def. Códigos

- Dados los alfabetos: Σ de los orígenes, y Γ para los códigos (desde ahora en adelante asumiremos $\Gamma = \{0, 1\}$). Un código es definido por la siguiente **función inyectiva**:

$$C : \Sigma \rightarrow \{0, 1\}^*$$

Donde $C(a)$, $a \in \Sigma$, es llamado el *código de a* .

- Note que es necesaria la inyección, de no ser así no sabríamos a que símbolo de Σ corresponde un código en particular.
- Sin embargo, al trabajar con códigos de largo variable, la inyección por si sola no es suficiente para tener un esquema de codificación robusto que univocamente decodifique un bitstream.

Códigos Univocamente Decodificable

Ej.1: $C(s_1) = 0$, $C(s_2) = 1$, $C(s_3) = 00$. ¿Cómo podríamos decodificar el bitstream 0000?

Códigos Univocamente Decodificable

Ej.1: $C(s_1) = 0$, $C(s_2) = 1$, $C(s_3) = 00$. ¿Cómo podríamos decodificar el bitstream 0000?

Una codificación se llama **univocamente decodificable** si esta no admite ambigüedad acerca de cual es la secuencia original al decodificar un bitstream.

Sea el sgte. esquema de codificación para secuencias de símbolos:

$$C' : \Sigma^* \rightarrow \{0, 1\}^* : s_1 s_2 \dots s_n \rightarrow C(s_1) C(s_2) \dots C(s_n)$$

en donde $C'(s_1 s_2 \dots s_n) = C(s_1) C(s_2) \dots C(s_n)$, entonces decimos que los códigos C son univocamente decodificable si y solo si C' es **inyectiva**.

Códigos Univocamente Decodificable

Ej.2 : Considere el siguiente esquema de codificación:

$C(s_1) = 1$, $C(s_2) = 10$, $C(s_3) = 00$.

- ¿Esto corresponde a una codificación univocamente decodificable?
- ¿Como se descomprime el bistring: 00100001000001?

Códigos Univocamente Decodificable

Ej.2 : Considere el siguiente esquema de codificación:

$C(s_1) = 1$, $C(s_2) = 10$, $C(s_3) = 00$.

- ¿Esto corresponde a una codificación univocamente decodificable?
- ¿Como se descomprime el bistring: 00100001000001?
- Para determinar la secuencia original debo contabilizar la cantidad de 0's que hay despues de cada 1.
- Si hay un número par de ceros ($2k$) entonces corresponde a s_1 seguido de k veces s_3 .
- Si hay un número impar de ceros ($2k + 1$) entonces corresponde a s_2 seguido de k veces s_3 .
- $decode(00100001000001) = s_3s_1s_3s_3s_2s_3s_3s_1$.

Decodificación Instantánea

Puede ocurrir que una codificación univocamente decodificable (como el ej. anterior), **no podamos decodificar el siguiente símbolo s en tiempo constante** o en un tiempo proporcional a largo de su código $|C(s)|$.

- Desde luego, es mucho más útil contar con una sistema de codificación que sea **instantáneo**.
- Este es el principio de los **Códigos de Prefijos**, en donde ningún código $C(s_i)$ es prefijo de otro código $C(s_j)$.
- Con dicha condición, eliminamos toda ambigüedad en la descompresión y el tiempo es constante o proporcional al largo de cada código.

Códigos Libres de Prefijos

Los **Códigos Libres de Prefijos** (*prefix codes*) son códigos **univocamente decodificable** y de **decodificación instantánea**.

Def. Una codificación C es ***prefix code*** si no existe algún código $C(s_i)$ que sea prefijo de otro código $C(s_j)$.

- Hemos visto anteriormente, que no toda codificación univocamente decodificable es prefix code.
- Sin embargo, $\forall$ codificación univocamente decodificable C que es no prefix code, siempre existe un prefix code que es al menos tan bueno como C .

Códigos de Shannon-Fano

- Este fue el primer esquema de codificación, para códigos libres de prefijos, en asignar códigos de largo cercano al óptimo.
- La entropía nos da $\lg(1/Pr(s))$ como largo óptimo para s .
- Shannon-Fano asigna $\lceil \lg(1/Pr(s)) \rceil$.
- Shannon-Fano ofrece códigos de largo promedio, $\bar{\ell}$, menor a un bit por sobre la entropía:

$$\bar{\ell} = \sum_{s \in \Sigma} Pr(s) \lceil \lg \frac{1}{Pr(s)} \rceil < \sum_{s \in \Sigma} Pr(s) \left(\lg \frac{1}{Pr(s)} + 1 \right) = \mathcal{H}(Pr) + 1$$

Algoritmo de Shannon-Fano

- 1- Los símbolos se ordenan descendientemente por su probabilidad.
- 2- Los símbolos se dividen en dos subconjuntos, los cuales suman igual probabilidad conjunta (o la más cerna posible).
- 3- A los códigos del primer subconjunto se le añade un 1, y un 0 a los del segundo subconjunto.
- 4- Prosiga recursivamente en cada subconjunto (ir al paso 2-) hasta que no queden más símbolos.

Algoritmo de Shannon-Fano

- 1- Los símbolos se ordenan descendientemente por su probabilidad.
- 2- Los símbolos se dividen en dos subconjuntos, los cuales suman igual probabilidad conjunta (o la más cerna posible).
- 3- A los códigos del primer subconjunto se le añade un 1, y un 0 a los del segundo subconjunto.
- 4- Prosiga recursivamente en cada subconjunto (ir al paso 2-) hasta que no queden más símbolos.

Ejemplo:

	$Pr(s)$	Divisiones	Final
1.	0.25	1 1	:11
2.	0.20	1 0	:10
3.	0.15	0 1 1	:011
4.	0.15	0 1 0	:010
5.	0.10	0 0 1	:001
6.	0.10	0 0 0 1	:0001
7.	0.05	0 0 0 0	:0000

Algoritmo de Shannon-Fano

- 1- Los símbolos se ordenan descendientemente por su probabilidad.
- 2- Los símbolos se dividen en dos subconjuntos, los cuales suman igual probabilidad conjunta (o la más cerna posible).
- 3- A los códigos del primer subconjunto se le añade un 1, y un 0 a los del segundo subconjunto.
- 4- Prosiga recursivamente en cada subconjunto (ir al paso 2-) hasta que no queden más símbolos.

Ejemplo:

	$Pr(s)$	Divisiones	Final
1.	0.25	1 1	:11
2.	0.20	1 0	:10
3.	0.15	0 1 1	:011
4.	0.15	0 1 0	:010
5.	0.10	0 0 1	:001
6.	0.10	0 0 0 1	:0001
7.	0.05	0 0 0 0	:0000

Ejercicio. Mediante la técnica de divide y vencerás, diseñe un pseudocódigo para construir los códigos de Shannon Fano, determine su relación de recurrencia y obtenga su complejidad. (Indicación: guíese por el algoritmo de quicksort).

Códigos de Huffman

- Huffman ofrece una codificación libre de prefijos que es óptima entre todos los esquemas que asignen un número entero de bits a cada código.
- Nunca pierde ante la codificación de Shannon-Fano.
- La cantidad de bits requeridos es entre $n\mathcal{H}(Pr)$ y $n(\mathcal{H}(Pr) + 1)$
- El algoritmo de Huffman corre en tiempo $\sigma \lg \sigma$, si mantenemos p.ej. un minheap sobre las probabilidades en los símbolos.
- Aunque el código más largo puede llegar a $\sigma - 1$, los códigos están acotados por $\mathcal{O}(\log \frac{1}{p_{min}})$, donde p_{min} es la probabilidad mínima entre los símbolos.
- Como $p_{min} \geq \frac{1}{n} \Rightarrow \mathcal{O}(\log \frac{1}{p_{min}}) = \mathcal{O}(\log n) \Rightarrow$ el algoritmo de Huffman puede codificar un asecuencia $S[1..n]$ en $\mathcal{O}(n)$, bajo el modelo RAM.

Árbol de Huffman

- El algoritmo de Huffman crea el llamado árbol de Huffman, el cual es usado para asignar los códigos a cada símbolo.
- Este es un árbol binario, donde sus σ hojas están asociadas a los σ símbolos del alfabeto.
- Cada arco e está etiquetado con un 0, si e va a un hijo izquierdo, o con un 1 si e va a un hijo derecho.
- El código para el símbolo s (asociado a la hoja σ_s) corresponde a la concatenación de las etiquetas en la ruta de la raíz a σ_s .
- El árbol tiende a ser balanceado si las probabilidades tienden a ser iguales (lo que es malo en términos de compresión)

Construcción del Árbol de Huffman

Algoritmo que crea el árbol T para $\Sigma[1..\sigma]$, donde los símbolos tienen frecuencias $f(1)...f(\sigma)$:

- 1.- Se crea una minheap $M[1..\sigma]$, asociando cada celda a un nodo de T . Inicialmente se tienen las σ hojas asociadas a las celdas de M , con sus respectivas probabilidades de ocurrencia.
- 2.- Se extraen dos elementos de M , u y v , y se inserta un nuevo elemento en el heap con probabilidad $f_{uv} = f(u) + f(v)$.
- 3.- Se crea un nuevo nodo interno en el árbol, i_{uv} , cuyos nodos izquierdo y derecho son u y v respectivamente; se asocia i_{uv} a la celda de M recién insertada con probabilidad f_{uv} .
- 4.- Se vuelve al paso (2) hasta que no hayan más elementos en M .
La complejidad de este algoritmo tiene un tiempo de $O(\sigma \log \sigma)$

Árbol de Huffman

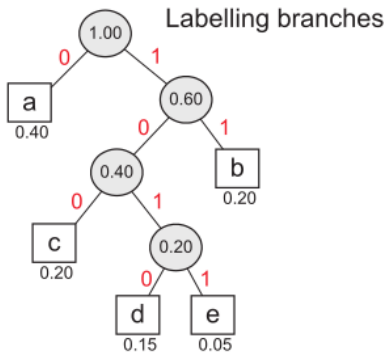
Ejemplo:

Construya el árbol de Huffman para el alfabeto $\Sigma = \{a, b, c, d, e\}$ con probabilidades de ocurencia $F = \{0.4, 0.2, 0.2, 0.15, 0.05\}$ respectivamente.

Árbol de Huffman

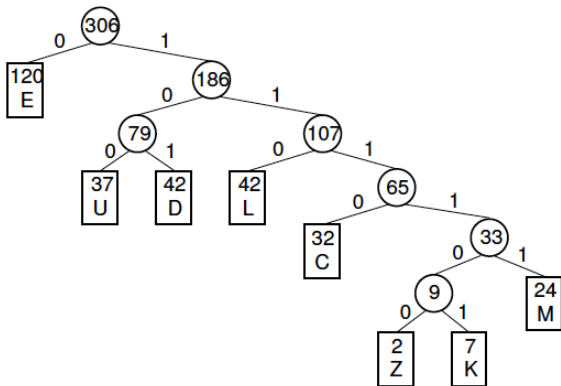
Ejemplo:

Construya el árbol de Huffman para el alfabeto $\Sigma = \{a, b, c, d, e\}$ con probabilidades de ocurencia $F = \{0.4, 0.2, 0.2, 0.15, 0.05\}$ respectivamente.



Árbol de Huffman

Otro Ejemplo:



Huffman Tree - Pseudocódigo

Input: El min-priority queue Q en base a las frecuencias de los σ elementos del alfabeto $\Sigma[1..\sigma]$. Q almacena nodos con los campos: *left*, *right* y *freq*.

Output: El árbol binario de Huffman con σ hojas.

```
buildHuffmanTree(Q){  
    while  $Q.size \geq 2$  do {  
         $z =$  nuevo nodo  
         $z.left = x = \text{ExtractMin}(Q)$   
         $z.right = y = \text{ExtractMin}(Q)$   
         $z.freq = x.freq + y.freq$   
         $Q.insert(z)$   
    }  
    return  $\text{ExtractMin}(Q)$   
}
```

Con un tiempo de $T_{Huff}(\sigma) = O(\sigma \log \sigma)$

El árbol binario construido es también llamado **árbol binario de Huffman con longitud mínima de camino**.

Códigos Canónicos de Huffman

- Muchos árboles de Huffman pueden obtenerse para una misma secuencia, basta intercambiar los hijos izquierdo y derecho en cualquier nodo interno.
- Lo realmente importante son los **largos de los códigos** que arroje el árbol de Huffman, ya que el resultado en términos de compresión será el mismo con cualquier árbol de Huffman.
- Los **Códigos Canónicos de Huffman** es el camino más adoptado, entre todas las posibilidades, ya que ayuda a almacenar el árbol de forma compacta y a una mejor decodificación.

Construcción del Árbol Canónico de Huffman

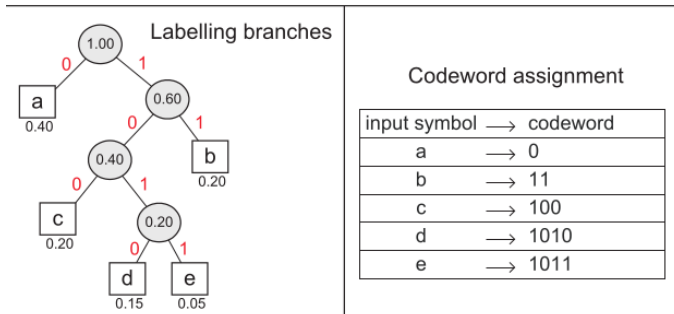
- Se requiere disponer de los largos que tendrán los códigos canónicos, los cuales vienen dados por cualquier árbol de Huffman para la secuencia de entrada.
- Se construye desde la hoja más izquierda avanzando hacia la del extremo derecho, e incrementando la profundidad si corresponde.
- Los códigos son asignados a los símbolos en orden creciente en sus largos.
- El primer código corresponde a una secuencia de ℓ_0 ceros, donde ℓ_0 corresponde al largo menor.
- Los códigos de un mismo largo corresponden a números binarios consecutivos.
- El primer código c_ℓ de largo ℓ esta relacionado con el último código para el largo $\ell - 1$, $c'_{\ell-1}$, por la ecuación: $c_\ell = 2(c'_{\ell-1} + 1)$.
Si no existe $c'_{\ell-1}$, entonces se agregan nodos intermedios hasta completar $\ell - 1$ niveles y c_ℓ corresponde a la hoja más a la izquierda posible que sea descendiente de alguno de estos nuevos nodos intermedios agregados.
- El árbol puede ser representado solo con los largos de los códigos, usando solo $\mathcal{O}(h)$ enteros, donde h corresponde a la altura del árbol de Huffman.

Árbol Canónico de Huffman

Por tanto, lo primero es obtener los largos de los códigos de cualquier árbol de Huffman. Ejemplo, para $\Sigma = \{a, b, c, d, e\}$, con ocurrencias $F = \{0.4, 0.2, 0.2, 0.15, 0.05\}$ tenemos:

Árbol Canónico de Huffman

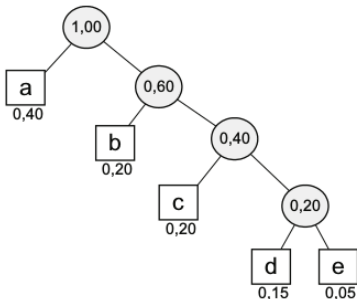
Por tanto, lo primero es obtener los largos de los códigos de cualquier árbol de Huffman. Ejemplo, para $\Sigma = \{a, b, c, d, e\}$, con ocurrencias $F = \{0.4, 0.2, 0.2, 0.15, 0.05\}$ tenemos:



Árbol Canónico de Huffman

Huffman Tree: $a \rightarrow 0$, $b \rightarrow 11$, $c \rightarrow 100$, $d \rightarrow 1010$, $e \rightarrow 1011$

Largos de códigos: $\{1, 2, 3, 4, 4\}$. Fórmula: $c_\ell = 2(c'_{\ell-1} + 1)$.



Canónica Tree: $a \rightarrow 0$, $b \rightarrow 10$, $c \rightarrow 110$, $d \rightarrow 1110$, $e \rightarrow 1111$

Códigos Canónicos de Huffman - Algoritmo de Decodificación

Construiremos un algoritmo intuitivo, usando tres arreglos:

- $\Sigma[1..\sigma]$, que almacenara los símbolos en orden ascendente.
- $F[1..h]$, donde $F[\ell] = i$ indica que el primer código con largo ℓ es $\Sigma[i]$.
- $C[1..h]$, donde $C[\ell]$ almacena el primer código de largo ℓ ($C[\ell] = \Sigma[F[\ell]]$).

Ahora deseamos decodificar los u símbolos de la secuencia comprimida S' .¹

¹Secuencialmente bit a bit, tarda $\mathcal{O}(|S'| + u \cdot h)$.

Códigos Canónicos de Huffman - Algoritmo de Decodificación

Construiremos un algoritmo intuitivo, usando tres arreglos:

- $\Sigma[1..\sigma]$, que almacenara los símbolos en orden ascendente.
- $F[1..h]$, donde $F[\ell] = i$ indica que el primer código con largo ℓ es $\Sigma[i]$.
- $C[1..h]$, donde $C[\ell]$ almacena el primer código de largo ℓ ($C[\ell] = \Sigma[F[\ell]]$).

Ahora deseamos decodificar los u símbolos de la secuencia comprimida S' .¹

1. Tomamos h bits de S' , $S'[i..i + h - 1]$, y los ponemos en un entero N (usando los h bits menos significativos de N)
2. Encontramos su largo ℓ , tal que $C[\ell] \cdot 2^{h-\ell} \leq N < C[\ell + 1] \cdot 2^{h-(\ell+1)}$ lo hacemos con búsqueda binaria en C en un tiempo de $\mathcal{O}(\log h)$.
3. Obtenemos N' , donde N' son los ℓ bits de más a la izquierda en $N[1..h]$.
4. Y arrojamos el símbolo $\Sigma[F[\ell] + N' - C[\ell]]$
5. Actualizamos $i = i + \ell$ y volvemos al paso 1.

¹Secuencialmente bit a bit, tarda $\mathcal{O}(|S'| + u \cdot h)$. 

Códigos Canónicos de Huffman - Algoritmo de Decodificación

Construiremos un algoritmo intuitivo, usando tres arreglos:

- $\Sigma[1..\sigma]$, que almacenara los símbolos en orden ascendente.
- $F[1..h]$, donde $F[\ell] = i$ indica que el primer código con largo ℓ es $\Sigma[i]$.
- $C[1..h]$, donde $C[\ell]$ almacena el primer código de largo ℓ ($C[\ell] = \Sigma[F[\ell]]$).

Ahora deseamos decodificar los u símbolos de la secuencia comprimida S' .¹

1. Tomamos h bits de S' , $S'[i..i + h - 1]$, y los ponemos en un entero N (usando los h bits menos significativos de N)
2. Encontramos su largo ℓ , tal que $C[\ell] \cdot 2^{h-\ell} \leq N < C[\ell + 1] \cdot 2^{h-(\ell+1)}$ lo hacemos con búsqueda binaria en C en un tiempo de $\mathcal{O}(\log h)$.
3. Obtenemos N' , donde N' son los ℓ bits de más a la izquierda en $N[1..h]$.
4. Y arrojamos el símbolo $\Sigma[F[\ell] + N' - C[\ell]]$
5. Actualizamos $i = i + \ell$ y volvemos al paso 1.

Así la decodificación de una secuencia de u símbolos codificados se logra en un tiempo de $\mathcal{O}(u \log h) = \mathcal{O}(u \log \log u)$.

¹Secuencialmente bit a bit, tarda $\mathcal{O}(|S'| + u \cdot h)$. 

Referencias

La mayoría de los contenidos referentes a la entropía fueron tomados del texto:

.- *Compact Data Structures. A Practical Aproach.*
Gonzalo Navarro. Cambrige University Press. 2016.